

Competitive Programming

Section 71 Editorial: Discrete Convexity And Lagrangian Optimization

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Monotone L1 Repair	4
2.1	How to find it	4
2.2	Correctness	4
2.3	Complexity and tests	4
2.4	C++	4
2.5	Python	5
3	B. Exact Red Spanning Tree	6
3.1	How to find it	6
3.2	Correctness	6
3.3	Complexity and tests	6
3.4	C++	6
3.5	Python	8
4	C. Massive Convex Allocation	10
4.1	How to find it	10
4.2	Correctness	10
4.3	Complexity and tests	10
4.4	C++	10
4.5	Python	12
5	D. Weighted Bounded Isotonic	14
5.1	Restatement and first reductions	14
5.2	Weighted slope representation	14
5.3	Correctness proof	14
5.4	Complexity and numeric range	15
5.5	Wrong answers and tests	15
5.6	C++	15
5.7	Python	16
6	Stress-testing summary	18

1 How to use this editorial

For each problem, identify the reduction before reading the code. The proof paragraph names the invariant that should have appeared in your own derivation.

2 A. Monotone L1 Repair

The prefix optimum as a function of its final value is convex. Its derivative changes only at stored breakpoints. Adding $|x-a|$ inserts x twice; truncating choices that violate monotonicity removes the largest left breakpoint and charges its displacement. Thus the heap maintains the exact compressed derivative.

2.1 How to find it

Start from the quadratic DP “best prefix cost when the final repaired value is x .” Its value as a function of x is convex and piecewise linear. Storing all values is unnecessary: a convex piecewise-linear function is determined by the ordered positions where its derivative changes.

2.2 Correctness

The max-heap stores the left breakpoints of the prefix value function. Adding $|x-a[i]|$ introduces two unit slope changes at $a[i]$. The monotonicity transition takes a prefix minimum over legal previous final values; in the derivative representation this removes the largest left breakpoint. If that breakpoint is y , moving it to $a[i]$ increases the minimum by $y-a[i]$. These are exactly the two pushes, one pop, and cost update in the code. Induction over prefixes therefore proves that the accumulated minimum equals the isotonic L1 optimum.

2.3 Complexity and tests

Each item performs three heap operations, so time is $O(n \log n)$ and memory is $O(n)$. Test already sorted, strictly decreasing, repeated values, negative values, and prefixes whose optimal medians are not unique.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    priority_queue<long long> left;
```

```

long long answer = 0;
for (int i = 0; i < n; ++i) {
    long long x;
    cin >> x;
    left.push(x);
    left.push(x);
    answer += left.top() - x;
    left.pop();
}
cout << answer << '\n';
}

```

2.5 Python

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    heap = []
    answer = 0
    for x in data[1:]:
        heapq.heappush(heap, -x)
        heapq.heappush(heap, -x)
        answer += -heapq.heappop(heap) - x
    print(answer)

if __name__ == "__main__":
    main()

```

3 B. Exact Red Spanning Tree

Price every red edge by `lambda`. Kruskal solves the penalized problem, and the red count is nonincreasing in `lambda`. Opposite tie rules give the extreme red counts among equally optimal trees. Matroid basis exchange fills the interval between them, so subtracting `lambda*k` recovers the exact-`k` objective.

3.1 How to find it

The difficult condition is the exact red count; everything else is an MST. Move that condition into the objective by charging `lambda` for each red edge. For a fixed charge the solver remains Kruskal, while increasing the charge can only decrease the red count of an optimum.

3.2 Correctness

Kruskal minimizes `weight + lambda*red`. Among equal modified weights, the two tie orders return the largest and smallest red counts of modified-optimal trees. The basis-exchange property of the graphic matroid connects optimal spanning trees by equal-cost exchanges, changing the red count one step at a time; consequently every count between those extremes is attainable at the same modified value. Binary search finds the last integer price whose red-preferring optimum still uses at least `k` reds. At that boundary `k` is in the optimal count interval, so subtracting `lambda*k` gives exactly the minimum original weight with `k` reds.

3.3 Complexity and tests

Each price evaluation sorts `m` edges, and $O(\log W)$ prices are tested, giving $O(m \log m \log W)$ here. Test impossible extreme counts, many equal modified weights, negative weights, all-one-color graphs, and cases with several MSTs.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (x != p[x])

```

```

        x = p[x];
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b)
            return false;
        if (sz[a] < sz[b])
            swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

struct Edge {
    int u, v, red;
    int64 w;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, wanted;
    if (!(cin >> n >> m >> wanted))
        return 0;
    vector<Edge> edges(m);
    for (auto &e : edges) {
        char c;
        cin >> e.u >> e.v >> e.w >> c;
        --e.u;
        --e.v;
        e.red = c == 'R';
    }

    auto run = [&](int64 lambda, bool prefer_red) {
        vector<int> order(m);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) {
            int64 a = edges[i].w + lambda * edges[i].red;
            int64 b = edges[j].w + lambda * edges[j].red;
            if (a != b)
                return a < b;
            return prefer_red ? edges[i].red > edges[j].red
                               : edges[i].red < edges[j].red;
        });
        DSU dsu(n);
        int used = 0, red = 0;
        int64 value = 0;
        for (int id : order)
            if (dsu.unite(edges[id].u, edges[id].v)) {
                ++used;
                red += edges[id].red;
            }
    };

```

```

        value += edges[id].w + lambda * edges[id].red;
    }
    return tuple<int, int, int64>{used, red, value};
};
auto [u1, min_red, z1] = run(400000000000000LL, false);
auto [u2, max_red, z2] = run(-400000000000000LL, true);
if (u1 != n - 1 || wanted < min_red || wanted > max_red) {
    cout << "IMPOSSIBLE\n";
    return 0;
}
int64 lo = -400000000000000LL, hi = 400000000000000LL;
while (lo < hi) {
    int64 mid = lo + (hi - lo + 1) / 2;
    auto [used, red, value] = run(mid, true);
    if (red >= wanted)
        lo = mid;
    else
        hi = mid - 1;
}
auto [used, red, modified] = run(lo, true);
cout << modified - lo * wanted << '\n';
}

```

3.5 Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(data)
    n, m, wanted = int(next(it)), int(next(it)), int(next(it))
    edges = []
    for _ in range(m):
        u, v, w, color = (
            int(next(it)) - 1,
            int(next(it)) - 1,
            int(next(it)),
            next(it),
        )
        edges.append((u, v, w, color == b"R"))

    def run(lam, prefer_red):
        order = sorted(
            edges,
            key=lambda e: (e[2] + lam * e[3], -e[3] if prefer_red else e[3]),

```

```

)
parent = list(range(n))
size = [1] * n

def find(x):
    while x != parent[x]:
        x = parent[x]
    return x

used = red = value = 0
for u, v, w, is_red in order:
    u, v = find(u), find(v)
    if u == v:
        continue
    if size[u] < size[v]:
        u, v = v, u
    parent[v] = u
    size[u] += size[v]
    used += 1
    red += is_red
    value += w + lam * is_red
return used, red, value

bound = 400_000_000_000_000
used, minimum, _ = run(bound, False)
_, maximum, _ = run(-bound, True)
if used != n - 1 or not minimum <= wanted <= maximum:
    print("IMPOSSIBLE")
    return
lo, hi = -bound, bound
while lo < hi:
    mid = (lo + hi + 1) // 2
    if run(mid, True)[1] >= wanted:
        lo = mid
    else:
        hi = mid - 1
_, _, modified = run(lo, True)
print(modified - lo * wanted)

if __name__ == "__main__":
    main()

```

4 C. Massive Convex Allocation

The j -th unit on a machine has marginal cost $a(2j-1)+b$, an increasing arithmetic progression. Binary-search the K -th smallest marginal globally. Take every marginal below the threshold and only as many tied threshold marginals as still needed.

4.1 How to find it

Expanding K units is impossible, but convexity makes the units ordered. The j -th unit on machine i costs $a[i](2j-1)+b[i]$; the problem is exactly to select the globally smallest K values from capped arithmetic progressions.

4.2 Correctness

Within one machine, selecting a later marginal without every earlier marginal can only increase cost, so every optimal selection is a prefix on each machine. Binary search finds the smallest threshold T with at least K available marginals at most T . All marginals below T must be selected: replacing one by a larger value cannot improve an optimum. Every remaining selected marginal can be chosen among values equal to T . The construction therefore selects exactly the global K smallest marginals, whose telescoping sum is the original quadratic objective.

4.3 Complexity and tests

One threshold count is $O(n)$ and 128-bit binary search uses $O(\log C)$ counts; memory is $O(n)$. Test $K=0$, insufficient capacity, negative first marginals, many ties at the threshold, and products beyond signed 64-bit range.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using i128 = __int128_t;
string show(i128 x) {
    if (x == 0)
        return "0";
    bool neg = x < 0;
    if (neg)
        x = -x;
```

```

string s;
while (x)
    s.push_back('0' + x % 10), x /= 10;
if (neg)
    s.push_back('-');
reverse(s.begin(), s.end());
return s;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    int64 k;
    if (!(cin >> n >> k))
        return 0;
    vector<int64> a(n), b(n), cap(n);
    int64 total = 0;
    for (int i = 0; i < n; ++i)
        cin >> a[i] >> b[i] >> cap[i], total += cap[i];
    if (total < k) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    auto count = [&](i128 threshold) {
        int64 result = 0;
        for (int i = 0; i < n; ++i) {
            if (threshold < a[i] + b[i])
                continue;
            int64 x = min<i128>(cap[i], (threshold - b[i] + a[i]) / (2 * a[i]));
            result += min(cap[i], x);
            if (result >= k)
                return k;
        }
        return result;
    };
    if (k == 0) {
        cout << "0\n";
        return 0;
    }
    i128 lo = 0, hi = 0;
    bool first = true;
    for (int i = 0; i < n; ++i)
        if (cap[i]) {
            i128 first_cost = (i128)a[i] + b[i];
            i128 last_cost = (i128)a[i] * (2 * (i128)cap[i] - 1) + b[i];
            if (first)
                lo = first_cost, hi = last_cost, first = false;
            else
                lo = min(lo, first_cost), hi = max(hi, last_cost);
        }
    while (lo < hi) {

```

```

    i128 mid = lo + (hi - lo) / 2;
    if (count(mid) >= k)
        hi = mid;
    else
        lo = mid + 1;
}
i128 threshold = lo, answer = 0;
int64 used = 0;
for (int i = 0; i < n; ++i) {
    int64 x = 0;
    if (threshold - 1 >= a[i] + b[i])
        x = min<i128>(cap[i], (threshold - 1 - b[i] + a[i]) / (2 * a[i]));
    used += x;
    answer += (i128)a[i] * x * x + (i128)b[i] * x;
}
answer += (i128)(k - used) * threshold;
cout << show(answer) << '\n';
}

```

4.5 Python

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, k = data[:2]
    machines = [tuple(data[i : i + 3]) for i in range(2, 3 * n + 2, 3)]
    if sum(cap for _, _, cap in machines) < k:
        print("IMPOSSIBLE")
        return

    def count(threshold):
        result = 0
        for a, b, cap in machines:
            if threshold >= a + b:
                result += min(cap, (threshold - b + a) // (2 * a))
            if result >= k:
                return k
        return result

    lo, hi = -4 * 10**18, 4 * 10**18
    while lo < hi:
        mid = (lo + hi) // 2
        if count(mid) >= k:
            hi = mid

```

```

        else:
            lo = mid + 1
        threshold = lo
        used = answer = 0
        for a, b, cap in machines:
            x = 0
            if threshold - 1 >= a + b:
                x = min(cap, (threshold - 1 - b + a) // (2 * a))
            used += x
            answer += a * x * x + b * x
        print(answer + (k - used) * threshold)

if __name__ == "__main__":
    main()

```

5 D. Weighted Bounded Isotonic

5.1 Restatement and first reductions

We need a nondecreasing integer sequence inside one global interval. Each target has a potentially enormous multiplicity $w[i]$, so expanding it into repeated unweighted targets is impossible.

First clamp every target to $[L, R]$. For any legal b , if $a < L$ then

$$|b-a| = (L-a) + |b-L|,$$

and the symmetric identity holds above R . Thus clamping adds a fixed cost and leaves an ordinary bounded target. Once all targets lie in the interval, the unbounded isotonic optimum also remains in their range, so it automatically respects the bounds.

5.2 Weighted slope representation

In the unweighted algorithm, target x inserts two breakpoint copies, removes one largest copy, and charges its displacement from x . Weight w scales the absolute-value derivative by w : insert breakpoint mass $2w$, remove largest mass w , and charge

$$(\text{removed_position} - x) * \text{removed_mass}.$$

A max-heap stores $(\text{position}, \text{mass})$. If only part of its maximum mass is removed, return the unused part to the heap. Because mass $2w$ at x was just inserted, every removed position is at least x , so every charged increment is nonnegative.

5.3 Correctness proof

Lemma 1. Clamping targets changes every feasible objective by exactly the fixed outside-interval cost accumulated by the algorithm.

Proof. The displayed identity holds for every legal chosen value, position by position. Summing with weights proves the claim. QED.

Lemma 2. After each prefix, the weighted heap and accumulated cost encode the exact convex optimum as a function of the last chosen value.

Proof. A weighted absolute value changes the derivative by $2w$ at its target, represented by inserting mass $2w$. Taking the prefix minimum required by nondecreasing order removes w units from the largest left breakpoints. Moving removed mass from position y to the new target x raises the minimum by $(y-x)$ per unit. This is exactly the heap transition. Induction over the prefix proves the invariant. QED.

By Lemma 2 the heap computes the optimum for clamped targets, and Lemma 1 adds back precisely the cost lost by clamping. Therefore the printed value is the minimum legal weighted objective.

5.4 Complexity and numeric range

Each inserted mass can be split once when it is partially removed; aggregate heap work is $O(n \log n)$ and memory is $O(n)$. Mass totals fit in signed 64-bit integers, but cost may not, so C++ uses `__int128`.

5.5 Wrong answers and tests

- Repeating a target w times.
- Inserting mass w rather than $2w$ before removing w .
- Clamping the eventual answer without charging the fixed outside distance.
- Using 64-bit total cost.
- Assuming the optimum is unique; only its minimum value is required.

Test one item below L , one above R , $L=R$, a sharply decreasing pair with opposite weight ratios, repeated targets, and weights near 10^9 .

5.6 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

string to_string128(int128 value) {
    if (value == 0) return "0";
    bool negative = value < 0;
    if (negative) value = -value;
    string result;
    while (value) {
        result.push_back(char('0' + value % 10));
        value /= 10;
    }
    if (negative) result.push_back('-');
    reverse(result.begin(), result.end());
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
```

```

int64 low, high;
cin >> n >> low >> high;

// Each pair is (breakpoint, multiplicity) in the derivative summary.
priority_queue<pair<int64, int64>> breakpoints;
int128 answer = 0;

for (int i = 0; i < n; ++i) {
    int64 target, weight;
    cin >> target >> weight;

    int64 x = clamp(target, low, high);
    answer += int128(weight) * llabs(target - x);

    breakpoints.push({x, 2 * weight});
    int64 remove = weight;
    while (remove > 0) {
        auto [position, mass] = breakpoints.top();
        breakpoints.pop();
        int64 take = min(remove, mass);
        answer += int128(position - x) * take;
        remove -= take;
        mass -= take;
        if (mass) breakpoints.push({position, mass});
    }
}

cout << to_string128(answer) << '\n';
}

```

5.7 Python

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, low, high = data[:3]
    heap = []
    answer = 0
    at = 3

    for _ in range(n):
        target, weight = data[at : at + 2]
        at += 2

```

```

x = min(high, max(low, target))
answer += weight * abs(target - x)

heapq.heappush(heap, (-x, 2 * weight))
remove = weight
while remove:
    negative_position, mass = heapq.heappop(heap)
    position = -negative_position
    take = min(remove, mass)
    answer += (position - x) * take
    remove -= take
    mass -= take
    if mass:
        heapq.heappush(heap, (-position, mass))

print(answer)

if __name__ == "__main__":
    main()

```

6 Stress-testing summary

The D oracle enumerates every nondecreasing sequence over a tiny bounded value domain and evaluates the weighted objective directly. It does not use heaps, derivatives, clamping identities, or convex optimization.